



PasswordStore Audit Report

Version 1.0

Fady Efat (Darkin)

April 14, 2026

PasswordStore Audit Report

Fady Efat (Darkin)

April 14, 2026

Prepared by: Fady Efat (Darkin) Lead Auditors: - Fady Efat (Darkin)

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
- Informational

Protocol Summary

A smart contract application for storing a password. Users should be able to store a password and then retrieve it later. Others should not be able to access the password.

Disclaimer

The Fady Efat (Darkin) team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
- Solc Version: 0.8.18
- Chain(s) to deploy contract to: Ethereum

Scope

```
./src/  
#-- PasswordStore.sol
```

Roles

- Owner: Should be able to store a password and then retrieve it later.
- Others: Should not be able to access the password.

Executive Summary

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Informational	1
Total	3

Findings

High

[H-1] Data Visibility: Storing passwords on-chain compromises privacy

Description

All data stored on the Ethereum blockchain is completely public and transparent, regardless of Solidity visibility modifiers. The `s_password` state variable is marked as `private`, but this only prevents *other smart contracts* from reading the variable directly.

Any external user, node operator, or off-chain script can easily inspect the contract's storage slots and read the plain text password.

Impact

The entire core functionality of the contract is broken. The password is meant to be a secret, but it is visible to the entire world. A malicious actor can read the private password and use it elsewhere or compromise the system relying on this secret.

Proof of Concept

In Solidity, state variables are stored in sequential storage slots starting from slot 0. - Slot 0: `s_owner` (address) - Slot 1: `s_password` (string)

Anyone can read the password directly from the blockchain using an RPC node and a tool like Foundry's `cast`:

```
# Read the storage slot 1 of the contract
cast storage <password_store_contract_address> 1 --rpc-url <your_rpc_url>
```

The console will output the hex-encoded string of the secret password.

Recommended Mitigation

Do not store sensitive or private data unencrypted on-chain.

If the architecture requires the blockchain to authenticate a password, you should hash the password off-chain (e.g., using `keccak256`) and only store the hash on-chain. When verifying, hash the user's input and compare it to the stored hash.

[H-2] Access Control: Missing authorization on `setPassword` allows anyone to change the password

Description

The `setPassword` function is declared as `external` but fails to implement any access control mechanisms.

Although the developer's NatSpec comment clearly states: `@notice This function allows only the owner to set a new password`, the code ignores this requirement. There is no `require` statement or `if` condition checking whether `msg.sender` (the caller) is the `s_owner`.

Impact

Any malicious user can call `setPassword` and overwrite the contract's password at any given time. This completely bypasses the intended authorization and allows attackers to hijack the password state.

Proof of Concept

The attack vector is extremely simple: 1. Alice (Owner) deploys the PasswordStore contract and sets her secure password. 2. Eve (Attacker) calls the setPassword ("EveIsTheNewOwner") function on Alice's contract. 3. The transaction successfully executes. 4. Alice's password is deleted and replaced with Eve's string without any authorization checks.

Code

```
function test_anyone_can_set_password(address randomaddress) public
↪ {
    vm.assume(randomaddress != owner);
    vm.prank(randomaddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);
    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEquals(actualPassword, expectedPassword);
}
```

Recommended Mitigation

Implement the same access control check found in the getPassword function. Require that the caller (msg.sender) is strictly the s_owner.

Apply the following modifications to PasswordStore.sol:

```
function setPassword(string memory newPassword) external {
+   if (msg.sender != s_owner) {
+       revert PasswordStore__NotOwner();
+   }
    s_password = newPassword;
    emit SetNetPassword();
}
```

Informational

[I-1] Documentation: Incorrect NatSpec documentation for getPassword function

Description

The NatSpec documentation accompanying the getPassword function incorrectly defines a parameter called newPassword. The getPassword function is a read-only (view) function that takes exactly **zero** parameters.

Impact

While this does not pose a direct security threat, misleading documentation can severely confuse other developers, auditors, and automated tools that parse NatSpec to generate UI/SDKs.

Proof of Concept

The codebase contains the following contradictory comments above getPassword:

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.  <--- Incorrect
↪ parameter
 */
function getPassword() external view returns (string memory) {
```

Recommended Mitigation

Remove the incorrect @param line from the NatSpec block to ensure the documentation perfectly aligns with the function signature.

```
/*
 * @notice This allows only the owner to retrieve the password.
-  * @param newPassword The new password to set.
 * /
function getPassword() external view returns (string memory) {
```